

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA

—o0o—



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: PHƯƠNG PHÁP SỐ CHO HỌC MÁY

**CÁC PHƯƠNG PHÁP TỐI ƯU CHUYÊN BIỆT
TRONG HỌC MÁY**

Nhóm 5

Kiều Thị Thu Trang MSSV: **24100093**

Trần Minh Sang MSSV: **24100012**

Ngô Quang Thiện MSSV:

Nguyễn Hưng Vũ MSSV:

Lớp: **K18-ICT3.24105.1**

GVHD: ThS. NCS. **ĐẶNG THỊ NGOAN**

HÀ NỘI - 2026

Mục lục

1	Giới thiệu chung	3
1.1	Đặt vấn đề	3
1.2	Ý tưởng chính	3
1.3	Phân loại các phương pháp tối ưu chuyên biệt	4
1.4	Phạm vi và mục tiêu của báo cáo	4
2	Bình phương tối thiểu phi tuyến	5
2.1	Bài toán hồi quy phi tuyến	5
2.2	Công thức tổng quát	6
2.3	Thuật toán Gauss-Newton	7
2.4	Thuật toán Levenberg-Marquardt	9
3	Bình phương tối thiểu có trọng số lặp	11
3.1	Công thức tổng quát và ý tưởng trọng số hóa	11
3.2	Ví dụ tối ưu chuẩn L^p	12
3.3	Ví dụ tối ưu L^1 và tính thưa	13
3.4	Thuật toán Weiszfeld cho trung vị hình học	14
3.5	Nhận xét về hội tụ và giới hạn của IRLS	15
4	Coordinate Descent và Alternation	15
4.1	Ý tưởng luân phiên tối ưu	15
4.2	Khi nào nên dùng alternation	16
4.3	Một số ví dụ tiêu biểu	16
4.4	Augmented Lagrangian và ADMM	19
5	Liên hệ với học máy	23
5.1	Gauss-Newton và Levenberg-Marquardt trong hồi quy phi tuyến	23
5.2	IRLS trong robust regression, Lasso và logistic regression . .	24
5.3	PCA và k-means như các bài toán alternation	25
5.4	ADMM trong tối ưu quy mô lớn và phân tán	26
5.5	Nhận xét chung	27

6 Thực nghiệm minh họa	27
6.1 Mục đích của phần thực nghiệm	27
6.2 Fit đường cong bằng Gauss-Newton/LM	28
6.3 IRLS cho hồi quy L^1	29
6.4 K-means hoặc ADMM cho nonnegative least-squares	30
6.5 So sánh tốc độ hội tụ	31
7 Kết luận	31
7.1 Tổng kết các phương pháp	31
7.2 Khi nào nên chọn phương pháp nào	32
7.3 Nhận xét cuối cùng	33
Tài liệu tham khảo	34

Danh mục hình ảnh

1 Giới thiệu chung

1.1 Đặt vấn đề

Trong các bài toán học máy, tối ưu hóa thường xuất hiện dưới dạng tìm một bộ tham số sao cho hàm mất mát đạt giá trị nhỏ nhất. Một cách tiếp cận tự nhiên là sử dụng các phương pháp tổng quát như Newton, quasi-Newton hoặc BFGS. Các phương pháp này có ưu điểm là có thể áp dụng cho nhiều hàm mục tiêu khác nhau, miễn là ta tính được đạo hàm bậc một, và trong một số trường hợp cả đạo hàm bậc hai.

Tuy nhiên, sự tổng quát này cũng đi kèm với một số hạn chế. Phương pháp Newton yêu cầu Hessian, tức ma trận đạo hàm bậc hai của hàm mục tiêu. Với mô hình có số biến lớn, việc lập, lưu trữ và giải hệ tuyến tính liên quan tới Hessian có thể rất tốn chi phí. BFGS và các biến thể quasi-Newton giảm bớt yêu cầu này bằng cách xấp xỉ Hessian từ lịch sử các bước lặp, nhưng ma trận xấp xỉ đó không chỉ phụ thuộc vào trạng thái hiện tại mà còn phụ thuộc vào toàn bộ quá trình lặp trước đó.

Trong khi đó, nhiều bài toán trong học máy không phải là một hàm mục tiêu bất kỳ. Chúng thường có cấu trúc rất rõ ràng: tổng bình phương các sai số, tổng các hàm trị tuyệt đối, các biến có thể tách thành từng khối, hoặc ràng buộc có dạng tuyến tính. Khai thác được các cấu trúc này giúp bài toán trở nên gọn hơn: thay vì gọi một bộ giải tối ưu tổng quát, ta chọn một phương pháp phù hợp với dạng của hàm mục tiêu và ràng buộc.

1.2 Ý tưởng chính

Ý tưởng xuyên suốt của báo cáo là: *một bài toán khó có thể trở nên dễ hơn nếu chọn đúng cách nhìn*. Với bài toán bình phương tối thiểu phi tuyến, cách nhìn phù hợp là tách hàm mất mát thành các residual rồi tuyến tính hóa chúng. Với các mục tiêu kiểu L^1 , trọng số thay đổi theo nghiệm hiện tại giúp đưa bài toán về gần dạng least-squares. Với các bài toán có nhiều nhóm biến, việc cố định một nhóm và tối ưu nhóm còn lại thường làm xuất hiện những bài toán con rất quen thuộc. Các chương sau sẽ đi từ ba kiểu cấu trúc này tới các thuật toán cụ thể.

1.3 Phân loại các phương pháp tối ưu chuyên biệt

Dựa trên cấu trúc bài toán, các phương pháp trong báo cáo được chia thành ba nhóm chính.

Bình phương tối thiểu phi tuyến. Các bài toán thuộc nhóm này có dạng

$$E(x) = \frac{1}{2} \sum_{i=1}^m f_i(x)^2, \quad (1.1)$$

trong đó $f_i(x)$ là các hàm dư. Nếu các f_i là tuyến tính, ta quay về bài toán least-squares cổ điển. Nếu các f_i là phi tuyến, Gauss-Newton và Levenberg-Marquardt là hai thuật toán tiêu biểu.

Tái trọng số. Khi hàm mục tiêu không phải tổng bình phương, ví dụ $\sum_i |r_i(x)|$ hoặc $\sum_i \|x - x_i\|_2$, nó vẫn có thể được viết dưới dạng gần giống bình phương tối thiểu:

$$E(x) = \sum_{i=1}^m w_i(x) g_i(x)^2. \quad (1.2)$$

Nếu tại bước lặp hiện tại xem $w_i(x)$ là cố định, bài toán còn lại là weighted least-squares. Cách làm này là cơ sở của IRLS.

Tách biến. Với bài toán có nhiều nhóm biến, hàm mục tiêu có thể được viết dưới dạng $f(x, y)$ hoặc $f(x, z)$, sau đó tối ưu từng nhóm biến một. Trong học máy, cách làm này xuất hiện trong PCA dưới dạng alternating least-squares, coordinate descent cho Lasso, và k-means clustering.

ADMM thuộc cùng mạch tư duy tách biến, nhưng có thêm ràng buộc. Biến phụ và ràng buộc tuyến tính được đưa vào để biến một bước tối ưu khó thành các cập nhật nhỏ hơn.

1.4 Phạm vi và mục tiêu của báo cáo

Báo cáo này tập trung vào các nội dung về *Nonlinear Least-Squares*, *Iteratively Reweighted Least-Squares*, *Coordinate Descent and Alternation* trong Chương 12 của sách *Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics* [10]. Cụ thể, báo cáo trình bày:

- bình phương tối thiểu phi tuyến, Gauss-Newton và LM;

- bình phương tối thiểu có trọng số lặp, bao gồm ví dụ chuẩn L^p , chuẩn L^1 và thuật toán Weiszfeld;
- coordinate descent, alternating optimization, augmented Lagrangian và ADMM;
- các liên hệ với học máy như curve fitting, robust regression, Lasso, logistic regression, PCA, k-means và tối ưu phân tán.

Mục tiêu của báo cáo không chỉ là trình bày lại lý thuyết trong tài liệu. Quan trọng hơn, báo cáo muốn làm rõ cách các thuật toán này xuất hiện trong những mô hình học máy quen thuộc. Chẳng hạn, k-means là một thuật toán alternation điển hình, PCA có thể được viết dưới dạng alternating least-squares, còn Lasso thường được giải bằng coordinate descent hoặc ADMM.

Hai câu hỏi sẽ được dùng xuyên suốt các chương tiếp theo: mỗi phương pháp đang tận dụng cấu trúc nào của bài toán, và cấu trúc đó xuất hiện ở đâu trong các thuật toán học máy quen thuộc?

2 Bình phương tối thiểu phi tuyến

2.1 Bài toán hồi quy phi tuyến

Một ví dụ cơ bản của bình phương tối thiểu phi tuyến là bài toán khớp đường cong. Giả sử các điểm dữ liệu $(t_1, y_1), (t_2, y_2), \dots, (t_m, y_m)$ đã được quan sát, mục tiêu là tìm hai tham số a, c sao cho mô hình

$$y = ce^{at} \tag{2.1}$$

khớp tốt với dữ liệu. Khi đó, sai số tại điểm thứ i là

$$r_i(a, c) = y_i - ce^{at_i}. \tag{2.2}$$

Một cách tự nhiên là chọn a, c sao cho tổng bình phương sai số nhỏ nhất:

$$E(a, c) = \sum_{i=1}^m (y_i - ce^{at_i})^2. \tag{2.3}$$

Nếu mô hình là tuyến tính theo tham số, ví dụ $y = \beta_0 + \beta_1 t$, bài toán có thể đưa về least-squares tuyến tính. Tuy nhiên, trong (2.1), tham số a nằm trong hàm mũ, nên bài toán trở thành phi tuyến. Việc lập một ma trận thiết kế rồi giải phương trình chuẩn tắc một lần không còn đủ; cần dùng một phương pháp lặp.

Bài toán trên xuất hiện rất nhiều trong học máy và khoa học dữ liệu: fitting hàm tăng trưởng, hiệu chỉnh mô hình vật lý, ước lượng tham số trong mô hình phi tuyến, hoặc huấn luyện những mô hình có đầu ra được mô tả bởi một hàm khả vi theo tham số.

2.2 Công thức tổng quát

Xét các hàm dư $f_1(x), f_2(x), \dots, f_m(x)$ với $x \in \mathbb{R}^n$. Mục tiêu của bài toán bình phương tối thiểu phi tuyến là làm cho tất cả các hàm dư này gần bằng 0. Hàm mục tiêu thường được viết dưới dạng

$$E_{\text{NLS}}(x) = \frac{1}{2} \sum_{i=1}^m f_i(x)^2. \quad (2.4)$$

Hệ số $\frac{1}{2}$ không làm thay đổi nghiệm tối ưu, nhưng giúp đạo hàm gọn hơn.

Đặt

$$F(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \\ \vdots \\ f_m(x) \end{bmatrix} \in \mathbb{R}^m, \quad (2.5)$$

và gọi $J(x) = DF(x) \in \mathbb{R}^{m \times n}$ là ma trận Jacobian của F . Khi đó,

$$E_{\text{NLS}}(x) = \frac{1}{2} \|F(x)\|_2^2. \quad (2.6)$$

Gradient của hàm mục tiêu có dạng

$$\nabla E_{\text{NLS}}(x) = J(x)^T F(x). \quad (2.7)$$

Hessian chính xác là

$$\nabla^2 E_{\text{NLS}}(x) = J(x)^T J(x) + \sum_{i=1}^m f_i(x) \nabla^2 f_i(x). \quad (2.8)$$

Công thức (2.8) giải thích vì sao bài toán NLS có cấu trúc đặc biệt. Nếu các residual $f_i(x)$ nhỏ gần nghiệm, thành phần $\sum_i f_i(x) \nabla^2 f_i(x)$ cũng nhỏ. Khi đó, $J(x)^T J(x)$ là một xấp xỉ hợp lý cho Hessian.

2.3 Thuật toán Gauss-Newton

Tuyến tính hóa các hàm dư

Giả sử điểm lặp hiện tại là x_k . Với mỗi residual f_i , xấp xỉ tuyến tính quanh x_k cho ta:

$$f_i(x_k + \delta) \approx f_i(x_k) + \nabla f_i(x_k)^T \delta. \quad (2.9)$$

Viết dưới dạng vector, ta có

$$F(x_k + \delta) \approx F(x_k) + J(x_k)\delta. \quad (2.10)$$

Thay xấp xỉ này vào (2.6), bài toán tại bước k trở thành

$$\min_{\delta \in \mathbb{R}^n} \frac{1}{2} \|F(x_k) + J(x_k)\delta\|_2^2. \quad (2.11)$$

Bài toán con này là least-squares tuyến tính theo δ .

Ở bước này chỉ có F được tuyến tính hóa. Tuy nhiên, do chuẩn ℓ_2 được bình phương, mô hình xấp xỉ vẫn là một hàm bậc hai theo δ . Vì thế, thông tin độ cong xuất hiện qua $J^T J$ dù thuật toán chỉ cần đạo hàm bậc một của các f_i .

Phương trình chuẩn tắc và công thức lặp

Điều kiện tối ưu của bài toán con (2.11) là

$$J(x_k)^T (F(x_k) + J(x_k)\delta_k) = 0. \quad (2.12)$$

Do đó,

$$J(x_k)^T J(x_k)\delta_k = -J(x_k)^T F(x_k). \quad (2.13)$$

Sau khi giải hệ tuyến tính (2.13), ta cập nhật

$$x_{k+1} = x_k + \delta_k. \quad (2.14)$$

Nếu $J(x_k)^T J(x_k)$ khả nghịch, công thức có thể viết hình thức là

$$x_{k+1} = x_k - \left(J(x_k)^T J(x_k) \right)^{-1} J(x_k)^T F(x_k). \quad (2.15)$$

Trong tính toán số, ta thường không nên tính ma trận nghịch đảo tường minh. Thay vào đó, ta giải hệ (2.13) bằng QR, Cholesky nếu đủ điều kiện, hoặc conjugate gradient khi kích thước lớn.

So sánh với Newton, Gauss-Newton thay Hessian chính xác (2.8) bằng xấp xỉ

$$H_{\text{GN}}(x_k) = J(x_k)^T J(x_k). \quad (2.16)$$

Nói cách khác, Gauss-Newton bỏ qua các đạo hàm bậc hai của từng residual f_i . So với Newton đầy đủ, bước lặp vì thế nhẹ hơn nhưng vẫn giữ lại phần độ cong quan trọng nhất của bài toán least-squares.

Ưu điểm, nhược điểm và điều kiện hội tụ

Gauss-Newton chỉ yêu cầu Jacobian, không cần Hessian đầy đủ. Mỗi bước lặp lại có dạng least-squares tuyến tính, nên các kỹ thuật tuyến tính ổn định như QR, Cholesky hoặc conjugate gradient có thể được tận dụng trực tiếp. Khi điểm khởi tạo đủ gần nghiệm và residual tại nghiệm nhỏ, tốc độ hội tụ có thể rất nhanh, gần với Newton trong các trường hợp thuận lợi [9].

Tuy nhiên, Gauss-Newton cũng có nhược điểm. Nếu điểm khởi tạo xa nghiệm, xấp xỉ tuyến tính (2.10) có thể kém chính xác. Nếu ma trận $J^T J$ gần suy biến, bước lặp có thể không ổn định. Ngoài ra, khi residual tối ưu không nhỏ, thành phần bị bỏ qua trong (2.8) có thể đáng kể, khiến xấp xỉ Gauss-Newton không còn tốt.

Với các bài toán curve fitting hoặc calibration có điểm khởi tạo tương đối hợp lý, Gauss-Newton thường là lựa chọn gọn và nhanh. Khi điểm khởi tạo chưa đủ tin cậy, Levenberg-Marquardt thường an toàn hơn.

2.4 Thuật toán Levenberg-Marquardt

Trust region

Gauss-Newton sử dụng xấp xỉ tuyến tính của F để chọn bước δ . Xấp xỉ này chỉ đáng tin trong một lân cận quanh x_k , nên độ dài bước lặp có thể được giới hạn bằng bài toán trust region:

$$\begin{aligned} \min_{\delta} \quad & \frac{1}{2} \|F(x_k) + J(x_k)\delta\|_2^2 \\ \text{subject to} \quad & \|\delta\|_2^2 \leq \Delta_k. \end{aligned} \quad (2.17)$$

Ở đây Δ_k là bán kính vùng tin cậy. Khi mô hình xấp xỉ dự đoán tốt sự giảm của hàm mục tiêu thật, vùng tin cậy được mở rộng. Nếu dự đoán sai, vùng này bị thu hẹp để bước sau thận trọng hơn.

Đặt

$$H_k = J(x_k)^T J(x_k), \quad g_k = J(x_k)^T F(x_k). \quad (2.18)$$

Bỏ qua hằng số không phụ thuộc vào δ , bài toán (2.17) tương đương

$$\begin{aligned} \min_{\delta} \quad & \frac{1}{2} \delta^T H_k \delta + g_k^T \delta \\ \text{subject to} \quad & \|\delta\|_2^2 \leq \Delta_k. \end{aligned} \quad (2.19)$$

Điều kiện KKT và tham số damping

Áp dụng điều kiện KKT cho (2.19), ta thu được điều kiện dừng

$$H_k \delta_k + g_k + \lambda_k \delta_k = 0, \quad (2.20)$$

với $\lambda_k \geq 0$. Do đó,

$$(H_k + \lambda_k I) \delta_k = -g_k. \quad (2.21)$$

Thay lại H_k và g_k , bước Levenberg-Marquardt có dạng

$$(J(x_k)^T J(x_k) + \lambda_k I) \delta_k = -J(x_k)^T F(x_k), \quad (2.22)$$

và

$$x_{k+1} = x_k + \delta_k. \quad (2.23)$$

Tham số λ_k được gọi là tham số damping. Khi $\lambda_k > 0$, ma trận $J^T J + \lambda_k I$ có xu hướng điều kiện tốt hơn $J^T J$, đặc biệt khi $J^T J$ gần suy biến.

Mối quan hệ với Gauss-Newton và gradient descent

Levenberg-Marquardt nằm giữa Gauss-Newton và gradient descent [6, 8]. Nếu λ_k rất nhỏ, hệ (2.22) gần với hệ Gauss-Newton:

$$J(x_k)^T J(x_k) \delta_k = -J(x_k)^T F(x_k). \quad (2.24)$$

Khi đó thuật toán tận dụng mạnh thông tin độ cong và có thể đi nhanh gần nghiệm.

Ngược lại, nếu λ_k rất lớn, ta có xấp xỉ

$$\delta_k \approx -\frac{1}{\lambda_k} J(x_k)^T F(x_k) = -\frac{1}{\lambda_k} \nabla E_{\text{NLS}}(x_k). \quad (2.25)$$

Khi đó bước lặp gần giống gradient descent với bước học nhỏ. Vì vậy, λ_k lớn giúp thuật toán thận trọng hơn khi chưa tin tưởng mô hình Gauss-Newton.

Điều chỉnh λ thích nghi

Tham số λ_k thường được điều chỉnh theo mức độ phù hợp giữa giảm thật và giảm dự đoán. Gọi

$$m_k(\delta) = \frac{1}{2} \|F(x_k) + J(x_k)\delta\|_2^2 \quad (2.26)$$

là mô hình xấp xỉ tại bước k . Tỷ số đánh giá chất lượng bước lặp là

$$\rho_k = \frac{E_{\text{NLS}}(x_k) - E_{\text{NLS}}(x_k + \delta_k)}{m_k(0) - m_k(\delta_k)}. \quad (2.27)$$

Nếu ρ_k lớn và dương, mô hình dự đoán tốt sự giảm của hàm mục tiêu thật; bước lặp được chấp nhận và λ_k được giảm để tiến gần Gauss-Newton hơn. Nếu ρ_k nhỏ hoặc âm, bước lặp không đáng tin; thuật toán giữ nguyên hoặc giảm tác động của bước đó, đồng thời tăng λ_k để chuyển về hướng thận trọng giống gradient descent.

Một quy tắc triển khai thường dùng có thể mô tả như sau:

- Giải hệ (2.22) để lấy δ_k .
- Tính ρ_k theo (2.27).

- Nếu ρ_k đủ lớn, đặt $x_{k+1} = x_k + \delta_k$ và giảm λ_k .
- Nếu ρ_k quá nhỏ, giữ nguyên x_k và tăng λ_k .

Nhờ cơ chế này, Levenberg-Marquardt thường ổn định hơn Gauss-Newton khi điểm khởi tạo chưa tốt, nhưng vẫn giữ được tốc độ nhanh khi đã đi vào vùng gần nghiệm.

3 Bình phương tối thiểu có trọng số lặp

3.1 Công thức tổng quát và ý tưởng trọng số hóa

IRLS là viết tắt của *Iteratively Reweighted Least-Squares*, tức bình phương tối thiểu có trọng số lặp. Ý tưởng của phương pháp là biến một bài toán không hoàn toàn thuộc dạng least-squares thành một chuỗi các bài toán least-squares có trọng số.

Xét hàm mục tiêu có dạng

$$E_{\text{IRLS}}(x) = \sum_{i=1}^m f_i(x) g_i(x)^2. \quad (3.1)$$

Trong biểu thức này, $f_i(x)$ đóng vai trò là trọng số của sai số bình phương $g_i(x)^2$. Nếu tại bước lặp thứ k trọng số được cố định ở

$$w_i^{(k)} = f_i(x_k), \quad (3.2)$$

thì bước tiếp theo được tính bằng bài toán

$$x_{k+1} = \arg \min_x \sum_{i=1}^m w_i^{(k)} g_i(x)^2. \quad (3.3)$$

Sau khi tìm được x_{k+1} , trọng số được cập nhật theo nghiệm mới rồi quá trình lặp tiếp tục.

Về mặt trực giác, IRLS cho phép mức ảnh hưởng của từng điểm dữ liệu thay đổi theo vòng lặp. Tùy mục tiêu ban đầu, những sai số lớn, sai số nhỏ hoặc thành phần gần bằng 0 sẽ được tăng hoặc giảm trọng số. Cơ chế này rất hợp với robust regression, sparse regression và một số bài toán thống kê cổ điển.

Nếu $g_i(x)$ là hàm tuyến tính, bài toán (3.3) là weighted least-squares tuyến tính. Nếu đặt

$$g_i(x) = a_i^T x - b_i, \quad (3.4)$$

với a_i^T là hàng thứ i của ma trận A , và $W^{(k)} = \text{diag}(w_1^{(k)}, \dots, w_m^{(k)})$, thì (3.3) trở thành

$$x_{k+1} = \arg \min_x (Ax - b)^T W^{(k)} (Ax - b). \quad (3.5)$$

Điều kiện tối ưu cho bài toán này là

$$A^T W^{(k)} A x_{k+1} = A^T W^{(k)} b. \quad (3.6)$$

3.2 Ví dụ tối ưu chuẩn L^p

Cho $A \in \mathbb{R}^{m \times n}$ và $b \in \mathbb{R}^m$. Bài toán least-squares tuyến tính cổ điển tối thiểu hóa $\|Ax - b\|_2^2$. Một tổng quát hóa tự nhiên là thay chuẩn ℓ_2 bằng chuẩn L^p :

$$E_p(x) = \|Ax - b\|_p^p = \sum_{i=1}^m |a_i^T x - b_i|^p. \quad (3.7)$$

Đặt $r_i(x) = a_i^T x - b_i$. Với $p > 0$,

$$|r_i(x)|^p = |r_i(x)|^{p-2} r_i(x)^2. \quad (3.8)$$

Vì vậy, nếu xem $|r_i(x)|^{p-2}$ là trọng số, bài toán L^p có dạng gần giống weighted least-squares.

Tại bước lặp k , một phiên bản ổn định của trọng số là

$$w_i^{(k)} = ((r_i(x_k))^2 + \varepsilon^2)^{\frac{p}{2}-1}, \quad (3.9)$$

trong đó $\varepsilon > 0$ là một số nhỏ để tránh chia cho 0. Bước kế tiếp giải

$$x_{k+1} = \arg \min_x \sum_{i=1}^m w_i^{(k)} (a_i^T x - b_i)^2. \quad (3.10)$$

Khi $p = 2$, trọng số bằng 1 và IRLS trở thành least-squares thông thường. Khi $p = 1$, công thức dẫn tới một phương pháp gần với tối ưu chuẩn L^1 . Với $p < 2$, các residual lớn thường bị giảm ảnh hưởng tương đối so với least-squares, do đó phương pháp có tính bền vững tốt hơn trước ngoại lai.

3.3 Ví dụ tối ưu L^1 và tính thừa

Trường hợp $p = 1$ đặc biệt quan trọng:

$$E_1(x) = \sum_{i=1}^m |a_i^T x - b_i|. \quad (3.11)$$

Về mặt hình thức,

$$|r_i(x)| = \frac{1}{|r_i(x)|} r_i(x)^2. \quad (3.12)$$

Do đó, tại bước k , ta chọn trọng số

$$w_i^{(k)} = \frac{1}{\max(|r_i(x_k)|, \delta)}, \quad (3.13)$$

với $\delta > 0$ để tránh chia cho 0. Bước cập nhật là

$$x_{k+1} = \arg \min_x \sum_{i=1}^m w_i^{(k)} (a_i^T x - b_i)^2. \quad (3.14)$$

So với least-squares, chuẩn L^1 ít nhạy hơn với ngoại lai vì sai số lớn chỉ tăng tuyến tính thay vì tăng bình phương.

Trong học máy, tính thừa thường được khuyến khích bằng chuẩn L^1 . Ví dụ, hồi quy Lasso có dạng

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \alpha \|x\|_1, \quad (3.15)$$

trong đó $\alpha > 0$ điều khiển mức phạt. Thành phần $\|x\|_1 = \sum_j |x_j|$ khiến nhiều hệ số x_j có xu hướng bằng 0, nhờ đó mô hình trở nên thừa và dễ diễn giải hơn [11]. IRLS có thể được dùng bằng cách xấp xỉ

$$|x_j| \approx \frac{x_j^2}{\max(|x_j^{(k)}|, \delta)}. \quad (3.16)$$

Khi đó, ở mỗi vòng lặp, bài toán Lasso được thay bằng một bài toán ridge regression có trọng số theo từng hệ số. Cách tiếp cận này không phải luôn là lựa chọn nhanh nhất cho Lasso, nhưng nó cho thấy rõ cách chuẩn L^1 có thể được xử lý thông qua một chuỗi bài toán bậc hai.

3.4 Thuật toán Weiszfeld cho trung vị hình học

Một ví dụ đẹp của IRLS là bài toán trung vị hình học. Cho các điểm $x_1, x_2, \dots, x_m \in \mathbb{R}^n$, bài toán cần tìm điểm x sao cho tổng khoảng cách Euclid tới các điểm đã cho là nhỏ nhất:

$$E(x) = \sum_{i=1}^m \|x - x_i\|_2. \quad (3.17)$$

Nếu dùng trung bình cộng, nghiệm sẽ nhảy với ngoại lai. Trung vị hình học bền vững hơn vì dùng tổng khoảng cách thay vì tổng bình phương khoảng cách.

Tương tự trường hợp L^1 ,

$$\|x - x_i\|_2 = \frac{1}{\|x - x_i\|_2} \|x - x_i\|_2^2. \quad (3.18)$$

Tại bước lặp k , đặt

$$w_i^{(k)} = \frac{1}{\max(\|x_k - x_i\|_2, \delta)}. \quad (3.19)$$

Bước IRLS trở thành

$$x_{k+1} = \arg \min_x \sum_{i=1}^m w_i^{(k)} \|x - x_i\|_2^2. \quad (3.20)$$

Bài toán (3.20) có nghiệm đóng. Lấy đạo hàm theo x :

$$\sum_{i=1}^m 2w_i^{(k)}(x - x_i) = 0. \quad (3.21)$$

Suy ra

$$x_{k+1} = \frac{\sum_{i=1}^m w_i^{(k)} x_i}{\sum_{i=1}^m w_i^{(k)}}. \quad (3.22)$$

Công thức này là dạng cơ bản của thuật toán Weiszfeld [12]. Mỗi vòng lặp vì thế chỉ cần tính một trung bình có trọng số; các điểm gần nghiệm hiện tại hơn nhận trọng số lớn hơn.

3.5 Nhận xét về hội tụ và giới hạn của IRLS

IRLS có ưu điểm là dễ xây dựng. Khi hàm mục tiêu có thể viết thành “trọng số nhân với bình phương”, một chiến lược trực tiếp là cố định trọng số rồi giải weighted least-squares. Với residual tuyến tính, mỗi bước lặp có thể giải bằng các kỹ thuật tuyến tính quen thuộc như Cholesky, QR hoặc conjugate gradient.

Tuy nhiên, IRLS không phải lúc nào cũng hội tụ tốt. Có ba vấn đề thường gặp:

- Nếu residual hoặc hệ số gần bằng 0, trọng số có thể rất lớn, làm hệ tuyến tính bị kém điều kiện.
- Nếu chọn δ hoặc ε quá lớn, bài toán được làm trơn quá mạnh và nghiệm thu được có thể lệch khỏi bài toán gốc.
- Nếu chọn δ quá nhỏ, thuật toán có thể dao động hoặc gặp khó khăn số học.

Với một số bài toán sparse recovery, hội tụ của IRLS đã được chứng minh khi thuật toán được điều chỉnh phù hợp [5]. Khi triển khai, vẫn cần theo dõi điều kiện dừng, chất lượng nghiệm và độ ổn định của hệ tuyến tính ở mỗi vòng lặp.

4 Coordinate Descent và Alternation

4.1 Ý tưởng luân phiên tối ưu

Giả sử ta cần tối thiểu hóa một hàm $f : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$. Thay vì xem biến đầu vào là một vector lớn, ta tách nó thành hai khối $x \in \mathbb{R}^n$ và $y \in \mathbb{R}^m$, rồi viết hàm mục tiêu dưới dạng $f(x, y)$. Một chiến lược đơn giản là cố định y để tối ưu x , sau đó cố định x để tối ưu y , và lặp lại:

$$\begin{cases} x_{k+1} = \arg \min_x f(x, y_k), \\ y_{k+1} = \arg \min_y f(x_{k+1}, y). \end{cases} \quad (4.1)$$

Chiến lược này được gọi là **alternating optimization**.

Nếu mỗi bài toán con được giải chính xác, giá trị hàm mục tiêu không tăng qua từng bước:

$$f(x_{k+1}, y_k) \leq f(x_k, y_k), \quad f(x_{k+1}, y_{k+1}) \leq f(x_{k+1}, y_k). \quad (4.2)$$

Suy ra

$$f(x_{k+1}, y_{k+1}) \leq f(x_k, y_k). \quad (4.3)$$

Tính giảm đơn điệu này là một ưu điểm lớn của alternation. Tuy nhiên, điều đó không đảm bảo thuật toán luôn đi tới nghiệm toàn cục. Với bài toán không lồi, alternation có thể hội tụ tới nghiệm cục bộ hoặc điểm dừng phụ thuộc mạnh vào khởi tạo.

4.2 Khi nào nên dùng alternation

Alternation thường phù hợp khi bài toán gốc khó, nhưng sau khi cố định một nhóm biến thì bài toán con trở nên đơn giản. Có hai tình huống đặc biệt hay gặp:

- Bài toán gốc phi tuyến theo toàn bộ biến, nhưng tuyến tính hoặc least-squares theo từng khối biến.
- Một số biến là biến rời rạc hoặc biến ràng buộc phức tạp, nhưng khi cố định các biến còn lại thì có công thức cập nhật đóng.

Trong học máy, alternation xuất hiện rất tự nhiên. Ví dụ, trong k-means, nếu biết tâm cụm thì việc gán nhãn cụm rất dễ; nếu biết nhãn cụm thì việc cập nhật tâm cụm cũng rất dễ. Nhưng nếu tối ưu đồng thời cả tâm cụm và nhãn cụm thì bài toán trở nên khó hơn nhiều.

4.3 Một số ví dụ tiêu biểu

PCA tổng quát và Alternating Least-Squares

Cho ma trận dữ liệu $X \in \mathbb{R}^{n \times m}$, trong đó mỗi cột là một điểm dữ liệu. PCA cổ điển tìm một không gian con chiều thấp sao cho dữ liệu được xấp xỉ tốt nhất trong chuẩn Frobenius. Một cách viết là

$$\min_{C, Y} \|X - CY\|_{\text{Fro}}^2, \quad (4.4)$$

trong đó $C \in \mathbb{R}^{n \times d}$ chứa các vector cơ sở và $Y \in \mathbb{R}^{d \times m}$ chứa hệ số biểu diễn của dữ liệu trong cơ sở đó.

Nếu cố định C , bài toán tối ưu Y là least-squares. Nếu cố định Y , bài toán tối ưu C cũng là least-squares. Vì vậy, ta có thuật toán alternating least-squares:

$$\begin{cases} Y_{k+1} = \arg \min_Y \|X - C_k Y\|_{\text{Fro}}^2, \\ C_{k+1} = \arg \min_C \|X - C Y_{k+1}\|_{\text{Fro}}^2. \end{cases} \quad (4.5)$$

Nếu thêm ràng buộc trực chuẩn cho các cột của C , ta quay về PCA cổ điển. Nếu thay chuẩn Frobenius bằng chuẩn khác, ví dụ chuẩn L^1 theo từng phần tử, ta thu được các biến thể robust PCA có khả năng giảm ảnh hưởng của nhiễu và ngoại lai [4].

Tích CY làm bài toán không lời nếu xét đồng thời C và Y . Khi một trong hai biến được cố định, phần còn lại giảm về một bài toán đơn giản hơn nhiều.

Bài toán ARAP

ARAP là viết tắt của *as-rigid-as-possible*, thường dùng trong xử lý hình học và biến dạng lưới. Một dạng năng lượng ARAP phẳng có thể viết như sau:

$$\min_{\{R_v\}, \{y_v\}} \sum_{v \in V} \sum_{(v,w) \in E} \|R_v(x_v - x_w) - (y_v - y_w)\|_2^2, \quad (4.6)$$

với ràng buộc

$$R_v^T R_v = I_2, \quad \forall v \in V. \quad (4.7)$$

Ở đây x_v là vị trí ban đầu, y_v là vị trí sau biến dạng, còn R_v là ma trận quay cục bộ tại đỉnh v .

Nếu tối ưu đồng thời tất cả R_v và y_v , bài toán rất khó vì có ràng buộc trực giao. Nhưng nếu tách biến, ta có hai bước:

- Cố định các R_v , tối ưu các y_v . Khi đó bài toán là least-squares thưa, có thể giải bằng hệ tuyến tính xác định dương.

- Cố định các y_v , tối ưu từng R_v . Các bài toán theo từng đỉnh tách rời nhau và trở thành bài toán Procrustes, có thể giải bằng SVD kích thước nhỏ.

Ví dụ này cho thấy sức mạnh của alternation: một bài toán phi tuyến lớn được thay bằng một bước giải hệ tuyến tính thưa và nhiều bài toán nhỏ có nghiệm đóng.

Coordinate descent cho least-squares

Coordinate descent là trường hợp cực đoan của alternation, trong đó mỗi lần chỉ tối ưu một tọa độ. Xét bài toán

$$\min_x \|Ax - b\|_2^2, \quad (4.8)$$

với $A = [a_1, a_2, \dots, a_n]$. Khai triển residual:

$$Ax - b = \sum_{j=1}^n x_j a_j - b. \quad (4.9)$$

Khi cố định tất cả các biến trừ x_i , điều kiện đạo hàm theo x_i bằng 0 là

$$a_i^T (Ax - b) = 0. \quad (4.10)$$

Suy ra công thức cập nhật

$$x_i \leftarrow \frac{a_i^T b - \sum_{j \neq i} x_j a_i^T a_j}{\|a_i\|_2^2}. \quad (4.11)$$

Thuật toán lặp qua $i = 1, 2, \dots, n$ nhiều lần cho đến khi hội tụ. Mỗi bước cập nhật chỉ thay đổi một tọa độ, nên chi phí từng bước rất nhỏ. Đặc điểm này đặc biệt hữu ích trong các bài toán có số chiều lớn hoặc có cấu trúc thưa.

Thuật toán k-means clustering

Cho các điểm dữ liệu $x_1, x_2, \dots, x_m \in \mathbb{R}^n$ và số cụm K . Thuật toán k-means tìm các tâm cụm $y_1, \dots, y_K$ sao cho tổng khoảng cách bình phương từ mỗi điểm tới tâm cụm gần nhất là nhỏ nhất:

$$E(y_1, \dots, y_K) = \sum_{i=1}^m \min_{c \in \{1, \dots, K\}} \|x_i - y_c\|_2^2. \quad (4.12)$$

Đặt c_i là chỉ số cụm của điểm x_i . Khi đó, bài toán có thể viết lại là

$$E(y_1, \dots, y_K; c_1, \dots, c_m) = \sum_{i=1}^m \|x_i - y_{c_i}\|_2^2. \quad (4.13)$$

K-means chính là alternation giữa hai bước [7]:

- **Bước gán cụm:** cố định các tâm y_j , chọn

$$c_i = \arg \min_{c \in \{1, \dots, K\}} \|x_i - y_c\|_2^2. \quad (4.14)$$

- **Bước cập nhật tâm:** cố định các c_i , cập nhật

$$y_j = \frac{\sum_{i:c_i=j} x_i}{|\{i : c_i = j\}|}. \quad (4.15)$$

Mỗi bước không làm tăng hàm mục tiêu. Tuy nhiên, vì bài toán không lồi, kết quả phụ thuộc vào khởi tạo. Một cách xử lý phổ biến là chạy k-means nhiều lần hoặc dùng k-means++ để chọn tâm ban đầu tốt hơn [1].

4.4 Augmented Lagrangian và ADMM

Từ hình phạt bậc hai đến nhân tử Lagrange

Xét bài toán có ràng buộc đẳng thức

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g(x) = 0. \end{aligned} \quad (4.16)$$

Một cách xử lý là dùng hình phạt bậc hai:

$$f_\rho(x) = f(x) + \frac{\rho}{2} \|g(x)\|_2^2. \quad (4.17)$$

Khi ρ lớn, nghiệm của bài toán phạt có xu hướng thỏa ràng buộc tốt hơn. Nhưng nếu ρ quá lớn, bài toán trở nên kém điều kiện và khó tối ưu.

Một cách khác là dùng Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T g(x). \quad (4.18)$$

Cách này đưa ràng buộc vào thông qua nhân tử Lagrange, nhưng bài toán trở thành bài toán yên ngựa: tối thiểu theo x và tối đa theo λ .

Chú ý 1 Một số tài liệu viết Lagrangian dưới dạng $f(x) - \lambda^T g(x)$ và cập nhật dual với dấu trừ. Hai cách viết này tương đương nếu thay λ bởi $-\lambda$. Trong báo cáo này, ta dùng quy ước dấu cộng để nhất quán với công thức ADMM ở các ví dụ phía sau.

Phương pháp Augmented Lagrangian

Augmented Lagrangian kết hợp hai ý tưởng trên:

$$\mathcal{L}_\rho(x, \lambda) = f(x) + \lambda^T g(x) + \frac{\rho}{2} \|g(x)\|_2^2. \quad (4.19)$$

Thuật toán lặp giữa cập nhật biến chính và cập nhật nhân tử:

$$\begin{cases} x_{k+1} = \arg \min_x \mathcal{L}_\rho(x, \lambda_k), \\ \lambda_{k+1} = \lambda_k + \rho g(x_{k+1}). \end{cases} \quad (4.20)$$

Thành phần phạt bậc hai giúp bước tối ưu theo x không đi quá xa khỏi tập ràng buộc, còn nhân tử Lagrange giúp không cần đưa ρ tiến tới vô hạn.

ADMM: công thức tổng quát

ADMM, tức *Alternating Direction Method of Multipliers*, áp dụng ý tưởng augmented Lagrangian cho bài toán có hai nhóm biến:

$$\begin{aligned} \min_{x,z} \quad & f(x) + h(z) \\ \text{subject to} \quad & Ax + Bz = c. \end{aligned} \quad (4.21)$$

Augmented Lagrangian tương ứng là

$$\mathcal{L}_\rho(x, z, \lambda) = f(x) + h(z) + \lambda^T (Ax + Bz - c) + \frac{\rho}{2} \|Ax + Bz - c\|_2^2. \quad (4.22)$$

ADMM cập nhật ba bước:

$$\begin{cases} x_{k+1} = \arg \min_x \mathcal{L}_\rho(x, z_k, \lambda_k), \\ z_{k+1} = \arg \min_z \mathcal{L}_\rho(x_{k+1}, z, \lambda_k), \\ \lambda_{k+1} = \lambda_k + \rho (Ax_{k+1} + Bz_{k+1} - c). \end{cases} \quad (4.23)$$

Khác với augmented Lagrangian cổ điển, ADMM không tối ưu đồng thời x và z . Thay vào đó, thuật toán tối ưu luân phiên từng biến. Việc tách này có thể làm tăng số vòng lặp, nhưng mỗi vòng lặp thường rẻ hơn nhiều và dễ song song hóa hơn [3].

Nhận xét về hội tụ và tham số ρ

Trong trường hợp f và h là các hàm lồi, ràng buộc là tuyến tính và augmented Lagrangian có điểm yên ngựa, ADMM có bảo đảm hội tụ tới nghiệm tối ưu toàn cục [3]. Với bài toán không lồi, ADMM vẫn thường được dùng trong thực nghiệm, nhưng lý thuyết hội tụ yếu hơn và kết quả có thể phụ thuộc vào cách tách biến cũng như khởi tạo.

Một đặc điểm của ADMM là thuật toán thường giảm nhanh sai số ở giai đoạn đầu, tức nhanh chóng tạo ra nghiệm xấp xỉ tốt, nhưng có thể cần nhiều vòng lặp để đạt độ chính xác rất cao. Vì vậy, trong một số hệ thống, ADMM được dùng để tìm nghiệm ban đầu tốt rồi chuyển sang một phương pháp tinh chỉnh khác nếu cần độ chính xác cuối rất chặt.

Tham số $\rho > 0$ điều khiển mức phạt đối với vi phạm ràng buộc. Nếu ρ quá nhỏ, biến primal có thể tiến chậm tới miền thỏa ràng buộc. Nếu ρ quá lớn, bước cập nhật có thể trở nên kém điều kiện hoặc quá tập trung vào ràng buộc mà giảm ít hàm mục tiêu. Trong triển khai, người ta thường theo dõi primal residual và dual residual để điều chỉnh ρ : nếu primal residual lớn hơn nhiều thì tăng ρ , còn nếu dual residual lớn hơn nhiều thì giảm ρ .

Ví dụ: Nonnegative least-squares bằng ADMM

Xét bài toán

$$\begin{aligned} \min_x \quad & \|Ax - b\|_2^2 \\ \text{subject to} \quad & x \geq 0. \end{aligned} \tag{4.24}$$

Ràng buộc $x \geq 0$ khiến least-squares tuyến tính thông thường không còn áp dụng trực tiếp. Đưa vào biến phụ z , bài toán được viết tương đương thành:

$$\begin{aligned} \min_{x,z} \quad & \|Ax - b\|_2^2 + h(z) \\ \text{subject to} \quad & x = z, \end{aligned} \tag{4.25}$$

trong đó

$$h(z) = \begin{cases} 0, & z \geq 0, \\ +\infty, & \text{ngược lại.} \end{cases} \quad (4.26)$$

Augmented Lagrangian là

$$\mathcal{L}_\rho(x, z, \lambda) = \|Ax - b\|_2^2 + h(z) + \lambda^T(x - z) + \frac{\rho}{2}\|x - z\|_2^2. \quad (4.27)$$

Bước cập nhật x thu được bằng cách cho gradient theo x bằng 0:

$$(2A^T A + \rho I)x_{k+1} = 2A^T b + \rho z_k - \lambda_k. \quad (4.28)$$

Do đó,

$$x_{k+1} = (2A^T A + \rho I)^{-1}(2A^T b + \rho z_k - \lambda_k). \quad (4.29)$$

Bước cập nhật z là phép chiếu lên miền không âm:

$$z_{k+1} = \max\left(x_{k+1} + \frac{1}{\rho}\lambda_k, 0\right), \quad (4.30)$$

trong đó phép max được hiểu theo từng phần tử. Cuối cùng,

$$\lambda_{k+1} = \lambda_k + \rho(x_{k+1} - z_{k+1}). \quad (4.31)$$

NNLS lúc này được tách thành ba thao tác rõ ràng: giải hệ tuyến tính, chiếu lên miền không âm và cập nhật biến dual.

Ví dụ: Geometric median bằng ADMM

Xét lại bài toán trung vị hình học:

$$\min_x \sum_{i=1}^N \|x - x_i\|_2. \quad (4.32)$$

Đặt biến phụ $z_i = x_i - x$, hay tương đương $z_i + x = x_i$, bài toán trở thành

$$\begin{aligned} \min_{x, \{z_i\}} \quad & \sum_{i=1}^N \|z_i\|_2 \\ \text{subject to} \quad & z_i + x = x_i, \quad i = 1, \dots, N. \end{aligned} \quad (4.33)$$

Với nhân tử λ_i cho từng ràng buộc, bước cập nhật x có dạng đóng:

$$x_{k+1} = \frac{1}{N} \sum_{i=1}^N \left(x_i - z_i^{(k)} - \frac{1}{\rho} \lambda_i^{(k)} \right). \quad (4.34)$$

Bước cập nhật z_i tách rời theo từng i . Đặt

$$u_i^{(k)} = x_i - x_{k+1} - \frac{1}{\rho} \lambda_i^{(k)}. \quad (4.35)$$

Khi đó,

$$z_i^{(k+1)} = \max \left(1 - \frac{1}{\rho \|u_i^{(k)}\|_2}, 0 \right) u_i^{(k)}. \quad (4.36)$$

Cập nhật này là phép co vector về phía 0, tương tự soft-thresholding nhưng theo chuẩn Euclid. Cuối cùng,

$$\lambda_i^{(k+1)} = \lambda_i^{(k)} + \rho(z_i^{(k+1)} + x_{k+1} - x_i). \quad (4.37)$$

Ví dụ này cho thấy lợi thế của ADMM: bài toán ban đầu không trơn tại các điểm $x = x_i$, nhưng sau khi tách biến, các bước cập nhật có dạng đơn giản và có thể song song hóa theo từng i .

Tóm lại, ADMM không chỉ là một thuật toán cụ thể mà còn là một khuôn mẫu thiết kế thuật toán. Khi gặp một bài toán khó, ta tìm cách tách nó thành các khối sao cho mỗi bước cập nhật có thể giải nhanh, có nghiệm đóng hoặc có phép chiếu đơn giản.

5 Liên hệ với học máy

5.1 Gauss-Newton và Levenberg-Marquardt trong hồi quy phi tuyến

Trong học máy, bài toán fitting mô hình thường có dạng tối thiểu hóa sai số giữa dự đoán và nhãn thật. Nếu mô hình dự đoán là $\hat{y}_i = f(t_i; \theta)$, với θ là vector tham số, thì residual là

$$r_i(\theta) = y_i - f(t_i; \theta). \quad (5.1)$$

Hàm mất mát bình phương là

$$E(\theta) = \frac{1}{2} \sum_{i=1}^m r_i(\theta)^2. \quad (5.2)$$

Hàm này có đúng dạng E_{NLS} trong Chương 2.

Với các mô hình hồi quy phi tuyến như mô hình mũ, mô hình logistic dạng đường cong, mô hình động lực học nhỏ hoặc bài toán calibration tham số, Gauss-Newton và Levenberg-Marquardt thường là lựa chọn tự nhiên. Gauss-Newton hiệu quả khi điểm khởi tạo đủ gần nghiệm. Levenberg-Marquardt ổn định hơn vì có tham số damping, nhờ đó có thể xử lý tốt hơn giai đoạn đầu khi nghiệm còn xa.

Trong mạng nơ-ron hiện đại, số tham số thường rất lớn nên việc giải hệ tuyến tính kiểu Gauss-Newton đầy đủ không phải lúc nào cũng thực tế. Tuy nhiên, ý tưởng dùng $J^T J$ làm xấp xỉ độ cong vẫn xuất hiện trong các phương pháp bậc hai xấp xỉ, natural gradient, hoặc các kỹ thuật Hessian-free cho bài toán least-squares lớn. Với loss dạng tổng bình phương residual, cấu trúc Jacobian trở thành thông tin trung tâm của bài toán.

5.2 IRLS trong robust regression, Lasso và logistic regression

Least-squares thông thường nhạy với ngoại lai vì sai số lớn bị bình phương. Trong robust regression, ta thường thay hàm mất mát bình phương bằng các hàm tăng chậm hơn, ví dụ chuẩn L^1 hoặc Huber loss. IRLS là một cách tiếp cận tự nhiên cho nhóm bài toán này: điểm nào có residual lớn có thể được gán trọng số nhỏ hơn ở vòng lặp tiếp theo.

Với hồi quy L^1 ,

$$\min_x \sum_{i=1}^m |a_i^T x - b_i|, \quad (5.3)$$

IRLS cập nhật trọng số

$$w_i^{(k)} = \frac{1}{\max(|a_i^T x_k - b_i|, \delta)} \quad (5.4)$$

rồi giải weighted least-squares. Khi residual lớn, trọng số nhỏ đi; do đó ngoại lai không chi phối nghiệm mạnh như trong least-squares.

Trong Lasso,

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \alpha \|x\|_1, \quad (5.5)$$

chuẩn L^1 trên tham số giúp mô hình có nghiệm thưa [11]. IRLS có thể xấp xỉ thành phần $\|x\|_1$ bằng một phạt bậc hai có trọng số thay đổi theo từng

vòng lặp. Dù vậy, Lasso thường được giải bằng coordinate descent hoặc ADMM vì hai phương pháp này khai thác tốt phép soft-thresholding.

Logistic regression cũng có một thuật toán IRLS cổ điển. Với dữ liệu nhị phân $y_i \in \{0, 1\}$ và

$$p_i = \frac{1}{1 + \exp(-a_i^T x)}, \quad (5.6)$$

negative log-likelihood là

$$\ell(x) = - \sum_{i=1}^m [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (5.7)$$

Newton cho logistic regression dẫn tới bài toán least-squares có trọng số với

$$W_{ii} = p_i(1 - p_i). \quad (5.8)$$

Vì vậy, thuật toán này thường được gọi là IRLS cho generalized linear models [2]. Ở đây, trọng số không đến từ chuẩn L^1 , mà đến từ Hessian của log-likelihood.

5.3 PCA và k-means như các bài toán alternation

PCA và k-means là hai thuật toán học máy rất quen thuộc, nhưng cả hai đều có thể được nhìn dưới góc độ alternation.

Với PCA dưới dạng matrix factorization,

$$X \approx CY, \quad (5.9)$$

có thể cố định C để tìm Y , rồi cố định Y để tìm C . Nếu dùng chuẩn Frobenius, cả hai bước đều là least-squares. Cách nhìn này hữu ích khi mô hình cần thêm ràng buộc hoặc thay đổi hàm mất mát, ví dụ nonnegative matrix factorization hoặc robust PCA.

Với k-means, biến cần tối ưu gồm tâm cụm $y_1, \dots, y_K$ và nhãn cụm $c_1, \dots, c_m$. Nếu biết tâm cụm, nhãn tối ưu của mỗi điểm là tâm gần nhất. Nếu biết nhãn, tâm tối ưu là trung bình cộng của các điểm trong cụm. Hai bước này chính là alternation:

$$c_i \leftarrow \arg \min_j \|x_i - y_j\|_2^2, \quad y_j \leftarrow \frac{1}{|\{i : c_i = j\}|} \sum_{i:c_i=j} x_i. \quad (5.10)$$

Điều này giải thích vì sao k-means đơn giản nhưng nhạy với khởi tạo. Mỗi bước làm giảm objective, nhưng objective không lồi nên thuật toán có thể dừng ở nghiệm cục bộ.

5.4 ADMM trong tối ưu quy mô lớn và phân tán

ADMM đặc biệt hữu ích trong các bài toán học máy quy mô lớn vì nó cho phép tách bài toán thành các phần nhỏ. Một ví dụ tiêu biểu là Lasso:

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \alpha \|x\|_1. \quad (5.11)$$

Đưa vào biến phụ z cùng ràng buộc $x = z$:

$$\begin{aligned} \min_{x,z} \quad & \frac{1}{2} \|Ax - b\|_2^2 + \alpha \|z\|_1 \\ \text{subject to} \quad & x = z. \end{aligned} \quad (5.12)$$

Khi đó, bước cập nhật x là một bài toán least-squares có điều chuẩn:

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho z_k - \lambda_k), \quad (5.13)$$

còn bước cập nhật z là soft-thresholding:

$$z_{k+1} = S_{\alpha/\rho} \left(x_{k+1} + \frac{1}{\rho} \lambda_k \right), \quad (5.14)$$

trong đó

$$S_\tau(u) = \text{sign}(u) \max(|u| - \tau, 0) \quad (5.15)$$

được áp dụng theo từng phần tử. Cuối cùng,

$$\lambda_{k+1} = \lambda_k + \rho(x_{k+1} - z_{k+1}). \quad (5.16)$$

Cách tách này rất có ý nghĩa: phần dữ liệu A, b nằm trong bước least-squares, còn phần thúc đẩy sparsity nằm trong bước soft-thresholding. Với dữ liệu lớn, các bước này có thể được tối ưu hóa riêng, tính toán song song hoặc phân tán trên nhiều máy [3].

Elastic Net cũng có thể được xử lý tương tự vì nó kết hợp phạt L^1 và L^2 . Phần L^2 làm hệ tuyến tính ổn định hơn, còn phần L^1 vẫn được xử lý bằng soft-thresholding. Nhìn chung, ADMM phù hợp khi bài toán gồm nhiều thành phần: một phần trơn, một phần không trơn, và một ràng buộc tuyến tính giúp tách hai phần đó.

5.5 Nhận xét chung

Các phương pháp trong báo cáo không nằm ngoài học máy; ngược lại, chúng xuất hiện ngay trong các thuật toán cơ bản:

- curve fitting và hồi quy phi tuyến sử dụng cấu trúc NLS;
- robust regression và logistic regression có thể dẫn tới IRLS;
- PCA, matrix factorization và k-means là các ví dụ tự nhiên của alternation;
- Lasso, Elastic Net và tối ưu phân tán thường dùng coordinate descent hoặc ADMM.

Các ví dụ trên cho thấy việc chọn thuật toán không nên tách rời cấu trúc của hàm mục tiêu. Khi nhận ra phần nào là least-squares, phần nào là chuẩn L^1 , phần nào có thể tách biến, ta có cơ sở để chọn một bộ giải gọn hơn thay vì dùng một phương pháp tổng quát cho mọi trường hợp.

6 Thực nghiệm minh họa

6.1 Mục đích của phần thực nghiệm

Phần này trình bày các thực nghiệm minh họa nhỏ nhằm kiểm tra trực giác của các phương pháp đã học và quan sát sự khác biệt giữa chúng trong một số bài toán đơn giản.

Trọng tâm của phần thực nghiệm không phải là xây dựng một hệ thống phần mềm lớn, mà là quan sát các hiện tượng sau:

- Gauss-Newton hội tụ nhanh khi khởi tạo tốt, còn LM ổn định hơn khi khởi tạo chưa tốt.
- IRLS cho hồi quy L^1 ít bị kéo lệch bởi ngoại lai hơn least-squares.
- K-means giảm objective sau mỗi vòng lặp nhưng phụ thuộc vào khởi tạo.

- ADMM tách một bài toán có ràng buộc thành các bước cập nhật đơn giản.

Các thí nghiệm được viết bằng Python, NumPy và Matplotlib, không dùng thư viện tối ưu chuyên dụng. Điều này phù hợp với mục tiêu của báo cáo: hiểu cấu trúc thuật toán trước khi dùng các bộ giải có sẵn.

Phần thực nghiệm gồm các nhóm minh họa chính:

- fit đường cong và biểu đồ hội tụ của Gauss-Newton/LM;
- hồi quy L^1 bằng IRLS khi dữ liệu có ngoại lai;
- quỹ đạo coordinate descent hai chiều;
- kết quả phân cụm và objective của k-means;
- hội tụ của ADMM cho nonnegative least-squares;
- minh họa hình phạt bậc hai trong augmented Lagrangian.

6.2 Fit đường cong bằng Gauss-Newton/LM

Xét dữ liệu sinh từ mô hình

$$y_i = c_{\text{true}} e^{a_{\text{true}} t_i} + \varepsilon_i, \quad (6.1)$$

trong đó ε_i là nhiễu nhỏ. Nhiệm vụ là ước lượng a và c từ dữ liệu. Residual được chọn là

$$f_i(a, c) = y_i - c e^{a t_i}. \quad (6.2)$$

Jacobian theo hai tham số a, c là

$$\frac{\partial f_i}{\partial a} = -c t_i e^{a t_i}, \quad \frac{\partial f_i}{\partial c} = -e^{a t_i}. \quad (6.3)$$

Với Gauss-Newton, tại mỗi vòng lặp ta giải

$$J(a_k, c_k)^T J(a_k, c_k) \delta_k = -J(a_k, c_k)^T F(a_k, c_k), \quad (6.4)$$

rồi cập nhật

$$\begin{bmatrix} a_{k+1} \\ c_{k+1} \end{bmatrix} = \begin{bmatrix} a_k \\ c_k \end{bmatrix} + \delta_k. \quad (6.5)$$

Với Levenberg-Marquardt, hệ tuyến tính được thay bằng

$$(J^T J + \lambda_k I) \delta_k = -J^T F. \quad (6.6)$$

Khi triển khai, ta nên chạy hai trường hợp khởi tạo:

- Khởi tạo gần nghiệm thật, ví dụ a_0 và c_0 chỉ lệch nhẹ so với tham số sinh dữ liệu.
- Khởi tạo xa nghiệm thật, ví dụ a_0 sai dấu hoặc c_0 quá nhỏ.

Kỳ vọng quan sát là Gauss-Newton rất nhanh khi khởi tạo gần nghiệm, nhưng có thể không ổn định khi khởi tạo xa. Levenberg-Marquardt thường đi chậm hơn lúc đầu nhưng ít bị bước nhảy quá lớn.

6.3 IRLS cho hồi quy L^1

Xét bài toán hồi quy tuyến tính

$$b = Ax + \varepsilon, \quad (6.7)$$

nhưng cố tình thêm một số ngoại lai vào vector b . Nếu dùng least-squares,

$$\min_x \|Ax - b\|_2^2, \quad (6.8)$$

các ngoại lai có thể kéo nghiệm lệch mạnh. Thay vào đó, ta dùng hồi quy L^1 :

$$\min_x \|Ax - b\|_1. \quad (6.9)$$

IRLS cập nhật theo các bước:

$$r^{(k)} = Ax_k - b, \quad w_i^{(k)} = \frac{1}{\max(|r_i^{(k)}|, \delta)}. \quad (6.10)$$

Sau đó giải

$$A^T W^{(k)} A x_{k+1} = A^T W^{(k)} b. \quad (6.11)$$

Chỉ số nên ghi lại gồm:

- giá trị $\|Ax_k - b\|_1$ qua từng vòng lặp;
- sai số giữa nghiệm ước lượng và nghiệm thật nếu dữ liệu được sinh mô phỏng;
- so sánh đường hồi quy least-squares và IRLS khi có ngoại lai.

Kỳ vọng là nghiệm L^1 ít bị lệch bởi các điểm ngoại lai hơn nghiệm least-squares.

6.4 K-means hoặc ADMM cho nonnegative least-squares

Có thể chọn một trong hai hướng thực nghiệm sau.

K-means. Dữ liệu hai chiều gồm K cụm được sinh mô phỏng, sau đó k-means được chạy với nhiều khởi tạo khác nhau. Hàm mục tiêu là

$$E = \sum_{i=1}^m \|x_i - y_{c_i}\|_2^2. \quad (6.12)$$

Sau mỗi vòng lặp, ta ghi lại E . Do mỗi bước gán cụm và cập nhật tâm đều tối ưu một phần của bài toán, E không tăng qua các vòng lặp. Tuy nhiên, các khởi tạo khác nhau có thể dẫn tới nghiệm cuối khác nhau.

ADMM cho nonnegative least-squares. Bài toán được xét là

$$\min_x \|Ax - b\|_2^2 \quad \text{subject to} \quad x \geq 0. \quad (6.13)$$

Với tách biến $x = z$, các bước ADMM là

$$x_{k+1} = (2A^T A + \rho I)^{-1} (2A^T b + \rho z_k - \lambda_k), \quad (6.14)$$

$$z_{k+1} = \max \left(x_{k+1} + \frac{1}{\rho} \lambda_k, 0 \right), \quad (6.15)$$

$$\lambda_{k+1} = \lambda_k + \rho(x_{k+1} - z_{k+1}). \quad (6.16)$$

Chỉ số nên theo dõi là primal residual

$$r_{\text{pri}}^{(k)} = \|x_k - z_k\|_2 \quad (6.17)$$

và giá trị objective $\|Ax_k - b\|_2^2$.

6.5 So sánh tốc độ hội tụ

Các thí nghiệm trên có thể được so sánh theo ba tiêu chí:

- **Số vòng lặp:** thuật toán cần bao nhiêu vòng để đạt điều kiện dừng.
- **Chi phí mỗi vòng:** mỗi vòng cần giải hệ tuyến tính, chiếu, cập nhật trọng số hay chỉ tính trung bình.
- **Độ ổn định:** thuật toán có nhạy với khởi tạo, ngoại lai hoặc tham số như λ , ρ , δ hay không.

Nhận xét kỳ vọng được tóm tắt như sau. Gauss-Newton thường nhanh khi đã ở gần nghiệm nhưng kém ổn định hơn Levenberg-Marquardt khi khởi tạo xa. IRLS dễ triển khai và bền vững với ngoại lai, nhưng có thể chậm hoặc kém điều kiện nếu trọng số quá lớn. K-means có mỗi vòng lặp rẻ, nhưng nhạy với tâm ban đầu. ADMM có thể cần nhiều vòng lặp, nhưng mỗi vòng thường tách thành các bước đơn giản, phù hợp với bài toán có ràng buộc hoặc thành phần không trơn.

Các kết quả thực nghiệm làm rõ những đánh đổi khó thấy nếu chỉ nhìn công thức: tốc độ hội tụ, độ ổn định, chi phí mỗi vòng lặp và độ nhạy với tham số.

7 Kết luận

7.1 Tổng kết các phương pháp

Báo cáo đã trình bày các phương pháp tối ưu chuyên biệt trong phạm vi các nội dung về bình phương tối thiểu phi tuyến, IRLS, coordinate descent, alternation và ADMM trong Chương 12 của sách [10]. Các phương pháp này không xử lý bài toán như một hàm mục tiêu hoàn toàn tổng quát; thay vào đó, mỗi phương pháp khai thác một cấu trúc riêng của bài toán.

Với bình phương tối thiểu phi tuyến, Gauss-Newton tuyến tính hóa các residual và giải một bài toán least-squares tại mỗi vòng lặp. Levenberg-Marquardt bổ sung tham số damping để điều chỉnh giữa Gauss-Newton và gradient descent, nhờ đó ổn định hơn khi điểm lặp còn xa nghiệm.

Với IRLS, các hàm mục tiêu như chuẩn L^1 , chuẩn L^p hoặc trung vị hình học được viết lại dưới dạng bình phương tối thiểu có trọng số. Mỗi vòng lặp gồm hai việc: cố định trọng số để giải least-squares, rồi cập nhật trọng số theo nghiệm mới.

Với coordinate descent và alternation, bài toán được chia thành các khối biến. Khi một khối được cố định, khối còn lại thường có bài toán con dễ hơn. Ý tưởng này dẫn tới nhiều thuật toán quen thuộc như alternating least-squares cho PCA, coordinate descent cho least-squares và k-means cho phân cụm.

Cuối cùng, ADMM mở rộng tinh thần tách biến sang bài toán có ràng buộc. Bằng cách đưa vào biến phụ và augmented Lagrangian, ADMM biến một bài toán khó thành các bước cập nhật đơn giản hơn, ví dụ giải hệ tuyến tính, chiếu lên miền không âm hoặc soft-thresholding.

7.2 Khi nào nên chọn phương pháp nào

Nếu hàm mục tiêu có dạng tổng bình phương residual và residual khả vi, Gauss-Newton là lựa chọn tự nhiên. Nếu điểm khởi tạo tốt và residual tại nghiệm nhỏ, thuật toán có thể hội tụ rất nhanh. Khi điểm khởi tạo chưa đáng tin hoặc ma trận $J^T J$ gần suy biến, Levenberg-Marquardt thường an toàn hơn.

Nếu bài toán liên quan tới chuẩn L^1 , robust regression hoặc một hàm mất mát có thể viết dưới dạng trọng số nhân với bình phương, IRLS là một lựa chọn đáng thử. Tuy nhiên, cần cẩn thận với tham số làm trơn như δ hoặc ε , vì chúng ảnh hưởng trực tiếp tới độ ổn định số học.

Nếu bài toán có nhiều nhóm biến và mỗi nhóm trở nên dễ tối ưu khi cố định các nhóm còn lại, alternation là một hướng tiếp cận hiệu quả. PCA, matrix factorization và k-means là các ví dụ điển hình. Nhược điểm chính là nghiệm có thể phụ thuộc vào khởi tạo do bài toán thường không lồi theo toàn bộ biến.

Nếu bài toán có ràng buộc tuyến tính hoặc gồm một phần trơn và một phần không trơn, ADMM là một công cụ mạnh. ADMM đặc biệt phù hợp

với Lasso, Elastic Net, nonnegative least-squares và các bài toán tối ưu phân tán. Dù vậy, việc chọn tham số ρ và tiêu chí dừng vẫn cần được kiểm tra thực nghiệm.

7.3 Nhận xét cuối cùng

Các phương pháp đã trình bày vẫn còn những giới hạn rõ ràng. Gauss-Newton và LM phụ thuộc vào chất lượng tuyến tính hóa; IRLS cần xử lý cẩn thận các trọng số gần suy biến; alternation và k-means nhạy với khởi tạo; ADMM lại phụ thuộc khá nhiều vào cách tách biến và lựa chọn ρ .

Một hướng mở rộng tự nhiên của báo cáo là kiểm tra các phương pháp này trên cùng một bộ dữ liệu với nhiều mức nhiễu và nhiều điểm khởi tạo khác nhau. Khi đó, phần so sánh không chỉ dừng ở công thức cập nhật, mà còn cho thấy rõ hơn phương pháp nào ổn định, phương pháp nào nhanh, và phương pháp nào phù hợp khi bài toán có ràng buộc hoặc thành phần không trơn.

Tài liệu tham khảo

- [1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.
- [2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.
- [4] Emmanuel J. Candès, Xiaodong Li, Yi Ma, and John Wright. Robust principal component analysis? *Journal of the ACM*, 58(3):11:1–11:37, 2011.
- [5] Ingrid Daubechies, Ronald DeVore, Massimo Fornasier, and C. Sinan Güntürk. Iteratively reweighted least squares minimization for sparse recovery. *Communications on Pure and Applied Mathematics*, 63(1):1–38, 2010.
- [6] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2):164–168, 1944.
- [7] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- [8] Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.
- [9] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006.

- [10] Justin Solomon. *Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics*. CRC Press, 2015.
- [11] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996.
- [12] Endre Weiszfeld. Sur le point pour lequel la somme des distances de n points donnés est minimum. *Tohoku Mathematical Journal*, 43:355–386, 1937.